

**Государственное бюджетное профессиональное образовательное учреждение
Республики Хакасия
«Хакасский политехнический колледж»**

**Документированный листинг.
Starseilor — аддон для Blender**

Студент группы ИС(ТП)-31

Горев А.П.

Дата: 09.06.2026

СОДЕРЖАНИЕ

1	Файл: <code>__init__.py</code>	3
2	Файл: <code>utils.py</code>	5
3	Файл: <code>properties.py</code>	14
4	Файл: <code>operators.py</code>	25
5	Файл: <code>panels.py</code>	35

1. Файл: __init__.py

Точка входа в аддон. Регистрирует все компоненты.

```
"""Starselor - аддон для Blender.
```

```
Размещение объектов вдоль кривой с поддержкой коллекций, диапазонов, случайных  
трансформаций.
```

```
"""
```

```
bl_info = {  
    "name": "Starselor",  
    "author": "Glue",  
    "version": (1, 5),  
    "blender": (4, 0, 0),  
    "location": "View3D > Toolbar > Starselor",  
    "description": "Размещение объектов вдоль кривой с настройками (коллекции для  
каждой кривой)",  
    "warning": "",  
    "wiki_url": "",  
    "category": "3D View",  
}
```

```
# Импорт внутренних модулей аддона
```

```
from . import utils          # Вспомогательные функции (создание коллекций, случайные  
значения)
```

```
from . import properties     # Классы свойств, хранящие настройки на объекте кривой
```

```
from . import operators      # Операторы: размещение, очистка, переключение ручек
```

```
from . import panels         # Панели интерфейса в боковой панели 3D View
```

```
def register():
```

```
    """Регистрация аддона в Blender."""
```

```
    properties.register()    # Регистрируем классы свойств и привязываем их к  
bpy.types.Object
```

```
    operators.register()     # Регистрируем операторы
```

```
    panels.register()        # Регистрируем панели UI
```

```
def unregister():
    """Отмена регистрации аддона с очисткой всех созданных экземпляров."""
    panels.unregister()
    operators.unregister()
    properties.unregister()
    utils.clear_all_instances() # Удаляем все коллекции Starselor и объекты в них

if __name__ == "__main__":
    register()
```

2. Файл: utils.py

Вспомогательные функции: работа с коллекциями, случайные трансформации, позиционирование на сегментах.

```
"""Утилиты Starselor: управление коллекциями, случайные числа, вычисление позиций  
вдоль сегмента."""
```

```
import bpy
import random
from math import pi
from mathutils import Vector, Euler, Quaternion, Matrix

# Константы
INSTANCE_COLLECTION_PREFIX = "Starselor_"          # Префикс для всех коллекций
аддона
RANDOM_SCALE_RANGE = (0.5, 1.5)                    # Диапазон случайного масштаба
RANDOM_ROTATION_RANGE = (-pi/2, pi/2)               # Диапазон случайного поворота
(радианы)

_random_cache = {}                                # Кэш для детерминированных
случайных значений (по seed)

def get_collection_name(curve_obj):
    """Генерирует уникальное имя коллекции на основе имени кривой."""
    if curve_obj:
        base_name = curve_obj.name.replace(".", "_").replace(" ", "_")
        return f"{INSTANCE_COLLECTION_PREFIX}{base_name}"
    return f"{INSTANCE_COLLECTION_PREFIX}Default"

def get_instance_collection(curve_obj):
    """Возвращает коллекцию для хранения экземпляров данной кривой.
    Если коллекции нет — создаёт и привязывает к сцене.
```

```

"""

coll_name = get_collection_name(curve_obj)
coll = bpy.data.collections.get(coll_name)
if coll is None:
    coll = bpy.data.collections.new(coll_name)
    if bpy.context.scene:
        bpy.context.scene.collection.children.link(coll)
    coll.hide_viewport = False
    coll.hide_render = False
    if curve_obj and hasattr(curve_obj, 'starselor_collection_name'):
        curve_obj.starselor_collection_name = coll_name
return coll

def clear_instances(curve_obj):
    """Удаляет все объекты из коллекции экземпляров для указанной кривой."""
    coll_name = get_collection_name(curve_obj)
    coll = bpy.data.collections.get(coll_name)
    if coll is None:
        return
    objects_to_remove = []
    try:
        objects_to_remove = [obj for obj in coll.objects if obj.name in
bpy.data.objects]
    except ReferenceError:
        # Обход устаревших ссылок
        for obj in bpy.data.objects:
            if any(col.name == coll_name for col in obj.users_collection if col):
                objects_to_remove.append(obj)
    if objects_to_remove:
        try:
            bpy.data.batch_remove(objects_to_remove)
            # Быстрое массовое
удаление
        except:

```

```

        for obj in objects_to_remove:                                # Резервный вариант
            try:
                if obj.name in bpy.data.objects:
                    bpy.data.objects.remove(obj, do_unlink=True)
            except:
                pass

def clear_all_instances():
    """Полная очистка всех коллекций Starselor и всех объектов в них."""
    collections_to_remove = []
    for coll in bpy.data.collections:
        if coll.name.startswith(INSTANCE_COLLECTION_PREFIX):
            objects_to_remove = []
            try:
                objects_to_remove = [obj for obj in coll.objects if obj.name in
bpy.data.objects]
            except:
                pass
            if objects_to_remove:
                try:
                    bpy.data.batch_remove(objects_to_remove)
                except:
                    for obj in objects_to_remove:
                        try:
                            if obj.name in bpy.data.objects:
                                bpy.data.objects.remove(obj, do_unlink=True)
                        except:
                            pass
                    collections_to_remove.append(coll)
    for coll in collections_to_remove:
        try:
            bpy.data.collections.remove(coll)
        except:

```

```

        pass

def _is_valid_context(context):
    """Проверяет, что текущий контекст позволяет безопасно выполнять размещение."""
    try:
        return (context.object and
                context.object.type == 'CURVE' and
                hasattr(context.object, 'main_object') and
                context.object.name in bpy.data.objects)
    except ReferenceError:
        return False

def _safe_place_objects(context):
    """Безопасно вызывает оператор размещения, игнорируя ошибки и отмены."""
    if _is_valid_context(context):
        try:
            if not getattr(bpy.app, 'is_undoing', False) and not getattr(bpy.app,
'is_redoing', False):
                bpy.ops.object.place_objects()
        except:
            pass

def _get_random_scale_factors(seed, lock_x=False, lock_y=False, lock_z=False):
    """Генерирует детерминированные случайные коэффициенты масштаба для заданного
seed.
Заблокированные оси получают коэффициент 1.0.
"""
    key = f"scale_{seed}"
    if key not in _random_cache:
        random.seed(seed)
        _random_cache[key] = [random.uniform(*RANDOM_SCALE_RANGE) for _ in range(3)]

```

```

factors = _random_cache[key].copy()
if lock_x:
    factors[0] = 1.0
if lock_y:
    factors[1] = 1.0
if lock_z:
    factors[2] = 1.0
return factors

def _get_random_rotation_factors(seed, lock_x=False, lock_y=False, lock_z=False):
    """Генерирует детерминированные случайные углы поворота (в радианах) для
    заданного seed."""
    key = f"rot_{seed}"
    if key not in _random_cache:
        random.seed(seed)
        _random_cache[key] = [random.uniform(*RANDOM_ROTATION_RANGE) for _ in
range(3)]
    factors = _random_cache[key].copy()
    if lock_x:
        factors[0] = 0.0
    if lock_y:
        factors[1] = 0.0
    if lock_z:
        factors[2] = 0.0
    return factors

def apply_transforms(obj, settings, obj_hash=None):
    """Применяет масштабирование и поворот к объекту на основе настроек.
    - obj_hash: уникальное число для детерминированной случайности (по умолчанию из
    имени).
    - Поддерживает: кастомный масштаб, случайный масштаб с блокировкой осей,
    кастомный поворот, случайный поворот с блокировкой.

```

```

"""

if obj_hash is None:
    obj_hash = hash(obj.name) % 10000

# --- Масштаб ---

if settings.NormalizationScaleObject:
    base_scale = Vector(settings.ScaleNormalizationObject)
else:
    base_scale = Vector((settings.ScaleObject,) * 3)

if settings.RandScaleObject:
    seed = settings.RandScaleObjectSlider + obj_hash
    rand_factors = Vector(_get_random_scale_factors(
        seed,
        lock_x=settings.RandScaleLockX,
        lock_y=settings.RandScaleLockY,
        lock_z=settings.RandScaleLockZ
    ))
    obj.scale = Vector(b * r for b, r in zip(base_scale, rand_factors))
else:
    obj.scale = base_scale

# --- Поворот ---

user_euler = Euler(settings.NormalizationRotateObject)

if settings.RandRotateObject:
    seed = settings.RandRoutateObjectSlider + obj_hash
    rand_rot = _get_random_rotation_factors(
        seed,
        lock_x=settings.RandRotLockX,
        lock_y=settings.RandRotLockY,
        lock_z=settings.RandRotLockZ
    )
    user_euler = Euler(b + r for b, r in zip(user_euler, rand_rot))
user_quat = user_euler.to_quaternion()

```

```

obj.rotation_mode = 'QUATERNION'

# Для edge-объектов: дополнительный поворот к направлению сегмента уже применён
до вызова,

# поэтому здесь мы просто умножаем кватернионы (добавляем пользовательский
поворот).

if 'edge' in settings.__class__.__name__.lower():
    obj.rotation_quaternion = obj.rotation_quaternion @ user_quat
else:
    obj.rotation_quaternion = user_quat

def get_segment_positions(p0_world, p1_world, distance=None, count=None,
                        offset_start=0.0, offset_end=0.0):
    """Вычисляет позиции для размещения объектов вдоль отрезка (p0 → p1) в мировых
    координатах.

    Параметры:
        p0_world, p1_world — конечные точки в мировом пространстве
        distance — шаг между объектами (если указан, приоритет над count)
        count — количество объектов (используется, если distance is None)
        offset_start, offset_end — отступы от начала и конца в мировых единицах

    Возвращает:
        (список позиций Vector, направление Vector) или ([], None) если невозможно.

    Логика:
        - Если указан distance: объекты распределяются равномерно с заданным шагом,
          центрируя группу на отрезке (последний шаг может быть меньше).
        - Если указан count: объекты делят отрезок на равные части,
          каждый объект располагается в центре своей части.
        - Если ничего не указано: возвращается одна точка в середине.
    """
    segment_vector = p1_world - p0_world

```

```

segment_length = segment_vector.length
if segment_length == 0 or (distance is not None and distance <= 0):
    return [], None

# Учитываем отступы
effective_start = offset_start
effective_end = segment_length - offset_end
if effective_start >= effective_end:
    return [], None

effective_length = effective_end - effective_start
positions = []
direction = segment_vector.normalized()

if distance is not None:
    # Размещение с фиксированным расстоянием
    if distance > effective_length:
        # Дистанция больше сегмента — ставим одну точку в центре
        mid_point = p0_world + direction * (effective_start + effective_length /
2.0)

        positions = [mid_point]
    else:
        num_objects = max(1, int(effective_length / distance))
        if num_objects == 1:
            mid_point = p0_world + direction * (effective_start +
effective_length / 2.0)

            positions = [mid_point]
        else:
            step = direction * distance
            remaining_space = effective_length - (distance * (num_objects - 1))
            start_offset = remaining_space / 2.0
            start_point = p0_world + direction * (effective_start + start_offset)
            positions = [start_point + step * i for i in range(num_objects)]

elif count is not None:

```

```

# Размещение с фиксированным количеством объектов

if count <= 0:
    return [], None

if count == 1:
    mid_point = p0_world + direction * (effective_start + effective_length /
2.0)

    positions = [mid_point]
else:
    step = effective_length / count
    start_point = p0_world + direction * effective_start
    positions = [start_point + direction * (step * (i + 0.5)) for i in
range(count)]

else:
    # По умолчанию: один объект в центре
    mid_point = p0_world + direction * (effective_start + effective_length / 2.0)
    positions = [mid_point]

return positions, direction

```

3. Файл: properties.py

Содержит все классы свойств, которые хранятся на объекте кривой, а также обработчики обновления.

```
"""Свойства аддона Starselor. Хранятся на объекте кривой и автоматически обновляют размещение."""
```

```
import bpy
from bpy.props import (
    PointerProperty, BoolProperty, IntProperty, FloatProperty,
    FloatVectorProperty, EnumProperty, StringProperty
)
from bpy.utils import register_class, unregister_class
from math import pi
from . import utils

# -----
# Вспомогательные функции-обработчики
# -----

def _create_update_handler():
    """Создаёт универсальный обработчик, который вызывает безопасное обновление
    размещения."""
    def handler(self, context):
        utils._safe_place_objects(context)
    return handler

# Обработчики для различных свойств — все они вызывают перерасположение
update_ObjectsInPoints = _create_update_handler()
update_ObjectsInEdges = _create_update_handler()
update_DelCurvePoints = _create_update_handler()
update_DelCurveEdges = _create_update_handler()
update_clip_settings = _create_update_handler()
update_equals_settings = _create_update_handler()
```

```
update_edge_spacing = _create_update_handler()
```

```
def update_curve(self, context):
```

```
    """Специальный обработчик при смене кривой: сохраняет оригинальное имя,  
    сбрасывает подразделение."""
```

```
    obj = context.object
```

```
    if not obj or obj.type != 'CURVE':
```

```
        return
```

```
    try:
```

```
        new_curve = self.curv
```

```
        if new_curve is None:
```

```
            self.curv_original_name = ""
```

```
            return
```

```
        self.curv_original_name = new_curve.name
```

```
        if hasattr(obj, 'curve_operftion'):
```

```
            obj.curve_operftion.SubdivideCurveSet = 0
```

```
        if obj.data != new_curve:
```

```
            utils.clear_instances(obj)
```

```
            obj.data = new_curve
```

```
            utils._safe_place_objects(context)
```

```
    except ReferenceError:
```

```
        pass
```

```
def update_subdivide(self, context):
```

```
    """Обработчик подразделения кривой: создаёт копию с дополнительными точками."""
```

```
    obj = context.object
```

```
    if not obj or obj.type != 'CURVE':
```

```
        return
```

```
    try:
```

```
        main_props = obj.main_object
```

```
        orig_name = main_props.curv_original_name
```

```
        if not orig_name:
```

```

        return

orig_curve = bpy.data.curves.get(orig_name)
if not orig_curve:
    return

level = self.SubdivideCurveSet
current_curve = obj.data
prev_active = context.view_layer.objects.active
prev_mode = context.mode if context.active_object else 'OBJECT'

# Если уже есть подразделённая кривая – удаляем её и возвращаем оригинал
if current_curve and current_curve.name != orig_name and
current_curve.name.startswith(orig_name + "_subdiv"):
    obj.data = orig_curve
    if current_curve.users == 0:
        bpy.data.curves.remove(current_curve)

if level == 0:
    obj.data = orig_curve
else:
    # Создаём копию кривой и подразделяем в режиме редактирования
    temp = orig_curve.copy()
    temp.name = f"{orig_name}_subdiv"
    obj.data = temp
    if obj.mode != 'EDIT':
        bpy.ops.object.mode_set(mode='EDIT')
    bpy.ops.curve.select_all(action='SELECT')
    if level > 0:
        bpy.ops.curve.subdivide(number_cuts=level)
    bpy.ops.object.mode_set(mode='OBJECT')

# Восстанавливаем активный объект и режим
if prev_active:
    context.view_layer.objects.active = prev_active
    if prev_mode == 'EDIT':

```

```

        bpy.ops.object.mode_set(mode='EDIT')

    utils._safe_place_objects(context)
except ReferenceError:
    pass

# -----
# Классы свойств (хранятся на bpy.types.Object)
# -----

class MainObject(bpy.types.PropertyGroup):
    """Основные свойства: какая кривая используется и какие объекты-шаблоны."""
    curv: PointerProperty(
        name='Curve',
        type=bpy.types.Curve,
        update=update_curve,
        description="Select curve to use"
    )
    curv_original_name: StringProperty(
        default="",
        description="Original curve name for subdivision tracking"
    )
    ObjectsInPoints: PointerProperty(
        name='Point Object',
        type=bpy.types.Object,
        update=update_ObjectsInPoints,
        description="Object to place at curve points"
    )
    ObjectsInEdges: PointerProperty(
        name='Edge Object',
        type=bpy.types.Object,
        update=update_ObjectsInEdges,
        description="Object to place along curve edges"
    )

```

)

```
class CurveOption(bpy.types.PropertyGroup):
    """Настройки кривой: подразделение и видимость групп объектов."""
    SubdivideCurveSet: IntProperty(
        name="Subdivide",
        default=0,
        min=0,
        soft_max=5,
        update=update_subdivide,
        description="Number of subdivision levels"
    )
    DelCurvePoints: BoolProperty(
        name="Hide Point Objects",
        default=False,
        update=update_DelCurvePoints,
        description="Hide objects placed at points"
    )
    DelCurveEdges: BoolProperty(
        name="Hide Edge Objects",
        default=False,
        update=update_DelCurveEdges,
        description="Hide objects placed along edges"
    )
)
```

```
class SegmentEnds(bpy.types.PropertyGroup):
    """Диапазон размещения вдоль кривой и клиппинг концов."""
    EqualLengths: BoolProperty(
        name="Use Range",
        default=False,
        update=update_equals_settings,
        description="Limit placement to specific range"
    )
)
```

```

)
StartEdge: FloatProperty(
    name="Start",
    default=0.0,
    min=0.0,
    max=1.0,
    subtype='FACTOR',
    update=update_equals_settings,
    description="Start of placement range (0=start, 1=end)"
)
EndEdge: FloatProperty(
    name="End",
    default=1.0,
    min=0.0,
    max=1.0,
    subtype='FACTOR',
    update=update_equals_settings,
    description="End of placement range"
)
ClipOn: BoolProperty(
    name="Enable Clip",
    default=False,
    update=update_clip_settings,
    description="Enable distance-based clipping from ends"
)
ClipDistance: FloatProperty(
    name="Distance",
    default=0.0,
    min=0.0,
    soft_max=10.0,
    subtype='DISTANCE',
    update=update_clip_settings,
    description="Distance to clip from each end (in Blender units)"
)

```

```

class EdgeSpacingProps(bpy.types.PropertyGroup):
    """Настройки распределения edge-объектов на каждом сегменте."""
    use_distance: BoolProperty(
        name="Use Distance",
        default=False,
        update=update_edge_spacing,
        description="Place objects at specified distance intervals"
    )
    use_count: BoolProperty(
        name="Use Count",
        default=False,
        update=update_edge_spacing,
        description="Place specified number of objects per segment"
    )
    edge_distance: FloatProperty(
        name="Distance",
        default=1.0,
        min=0.001,
        soft_max=100.0,
        subtype='DISTANCE',
        update=update_edge_spacing,
        description="Distance between edge objects"
    )
    edge_count: IntProperty(
        name="Count",
        default=1,
        min=1,
        soft_max=100,
        update=update_edge_spacing,
        description="Number of objects per segment"
    )
    offset_start: FloatProperty(

```

```

        name="Offset Start",
        default=0.0,
        min=0.0,
        soft_max=10.0,
        subtype='DISTANCE',
        update=update_edge_spacing,
        description="Offset from segment start"
    )
    offset_end: FloatProperty(
        name="Offset End",
        default=0.0,
        min=0.0,
        soft_max=10.0,
        subtype='DISTANCE',
        update=update_edge_spacing,
        description="Offset from segment end"
    )

```

```

class OptionBase(bpy.types.PropertyGroup):
    """Базовые настройки трансформаций (масштаб, поворот, случайные вариации)."""
    ScaleObject: FloatProperty(name="Uniform Scale", default=1.0, min=0.0)
    NormalizationScaleObject: BoolProperty(name="Use Custom Scale", default=False)
    ScaleNormalizationObject: FloatVectorProperty(
        name="Custom Scale", size=3, min=0.0, default=(1.0, 1.0, 1.0), subtype='XYZ'
    )
    RandScaleObject: BoolProperty(name="Random Scale", default=False)
    RandScaleObjectSlider: IntProperty(name="Seed", default=0, min=0)
    RandScaleLockX: BoolProperty(name="X", default=False,
update=_create_update_handler())
    RandScaleLockY: BoolProperty(name="Y", default=False,
update=_create_update_handler())
    RandScaleLockZ: BoolProperty(name="Z", default=False,
update=_create_update_handler())

```

```

NormalizationRotateObject: FloatVectorProperty(
    name="Rotation", subtype='EULER', size=3, min=-2*pi, max=2*pi, default=(0.0,
0.0, 0.0)
)
RandRotateObject: BoolProperty(name="Random Rotation", default=False)
RandRotateObjectSlider: IntProperty(name="Seed", default=0, min=0)
RandRotLockX: BoolProperty(name="X", default=False,
update=_create_update_handler())
RandRotLockY: BoolProperty(name="Y", default=False,
update=_create_update_handler())
RandRotLockZ: BoolProperty(name="Z", default=False,
update=_create_update_handler())

```

```

class PointOptionSettings(OptionBase):
    """Настройки для point-объектов (наследует OptionBase)."""
    pass

```

```

class EdgeOptionSettings(OptionBase):
    """Настройки для edge-объектов (наследует OptionBase)."""
    pass

```

```

class OptionContainerGroup(bpy.types.PropertyGroup):
    """Контейнер, позволяющий переключаться между настройками point и edge."""
    select_target: EnumProperty(
        name="Target",
        description="Select which object type to configure",
        items=[
            ("2", "Point Object", "Settings for objects placed at points"),
            ("3", "Edge Object", "Settings for objects placed along edges"),
        ],
    ),

```

```

        default="2"
    )
    point: PointerProperty(type=PointOptionSettings)
    edge: PointerProperty(type=EdgeOptionSettings)

# -----
# Регистрация свойств и привязка к bpy.types.Object
# -----

classes = [
    MainObject,
    CurveOption,
    SegmentEnds,
    EdgeSpacingProps,
    OptionBase,
    PointOptionSettings,
    EdgeOptionSettings,
    OptionContainerGroup,
]

def register():
    for cl in classes:
        register_class(cl)

    # Привязываем экземпляры свойств к каждому объекту Blender
    bpy.types.Object.main_object = PointerProperty(type=MainObject)
    bpy.types.Object.curve_option = PointerProperty(type=CurveOption)
    bpy.types.Object.curve_equals = PointerProperty(type=SegmentEnds)
    bpy.types.Object.curve_option_container =
PointerProperty(type=OptionContainerGroup)
    bpy.types.Object.edge_spacing_props = PointerProperty(type=EdgeSpacingProps)
    bpy.types.Object.starsel_collection_name = StringProperty(default="")

def unregister():

```

```

# Удаляем привязки с объектов
for prop in ['main_object', 'curve_operftion', 'curve_equals',
             'curve_option_container', 'edge_spacing_props',
'starselor_collection_name']:
    try:
        delattr(bpy.types.Object, prop)
    except:
        pass
for cl in reversed(classes):
    try:
        unregister_class(cl)
    except:
        pass

```

4. Файл: operators.py

Содержит основные операторы: размещение, очистка, переключение ручек.

```
"""Операторы Starselor. Выполняют размещение объектов, очистку, вспомогательные действия."""
```

```
import bpy

from bpy.utils import register_class, unregister_class
from mathutils import Vector
from . import utils
from .properties import (
    MainObject, CurveOption, SegmentEnds, EdgeSpacingProps,
    PointOptionSettings, EdgeOptionSettings, OptionContainerGroup
)

class CURVE_OT_toggle_handle_type(bpy.types.Operator):
    """Переключает тип ручек всех точек кривой между AUTO и VECTOR."""
    bl_idname = "curve.toggle_handle_type"
    bl_label = "Toggle Handle Type"
    bl_description = "Переключить тип ручек между Automatic и Vector"
    bl_options = {'REGISTER', 'UNDO'}

    def execute(self, context):
        obj = context.object

        if not obj or obj.type != 'CURVE':
            self.report({'WARNING'}, "Active object is not a curve")
            return {'CANCELLED'}

        curve = obj.data

        if curve is None:
            self.report({'WARNING'}, "No curve data")
            return {'CANCELLED'}

        # Определяем текущий тип по первой точке
```

```

current_type = None
for spline in curve.splines:
    if spline.type == 'BEZIER' and spline.bezier_points:
        current_type = spline.bezier_points[0].handle_left_type
        break

new_type = 'VECTOR' if current_type == 'AUTO' else 'AUTO'

for spline in curve.splines:
    if spline.type == 'BEZIER':
        for bezier_point in spline.bezier_points:
            bezier_point.handle_left_type = new_type
            bezier_point.handle_right_type = new_type

self.report({'INFO'}, f"Handles set to {new_type}")
return {'FINISHED'}

class OBJECT_OT_place_objects(bpy.types.Operator):
    """Главный оператор: размещает point- и edge-объекты вдоль кривой."""
    bl_idname = "object.place_objects"
    bl_label = "Place Objects on Curve"
    bl_description = "Create instances of Point and Edge objects along the curve"
    bl_options = {'REGISTER', 'UNDO'}

    def execute(self, context):
        obj = context.object
        if not obj or not hasattr(obj, 'main_object'):
            self.report({'WARNING'}, "Active object must be a curve with main
properties")
            return {'CANCELLED'}

        main_props = obj.main_object
        curve_opt = obj.curve_operftion

```

```

curve_data = obj.data
if not curve_data:
    self.report({'WARNING'}, "Curve has no data")
    return {'CANCELLED'}

# Получаем объекты-шаблоны
point_template = main_props.ObjectsInPoints
edge_template = main_props.ObjectsInEdges
if point_template and point_template.name not in bpy.data.objects:
    point_template = None
if edge_template and edge_template.name not in bpy.data.objects:
    edge_template = None

point_settings = obj.curve_option_container.point
edge_settings = obj.curve_option_container.edge
seg_props = obj.curve_equals
edge_spacing = obj.edge_spacing_props

need_points = point_template and not curve_opt.DelCurvePoints
need_edges = edge_template and not curve_opt.DelCurveEdges

if not (need_points or need_edges):
    self.report({'WARNING'}, "No objects to place. Select Point Object or
Edge Object first.")
    return {'CANCELLED'}

# Очищаем старые экземпляры
utils.clear_instances(obj)
matrix_world = obj.matrix_world
coll = utils.get_instance_collection(obj)

start_f = seg_props.StartEdge
end_f = seg_props.EndEdge
if seg_props.EqualLengths and start_f > end_f:

```

```

        start_f, end_f = end_f, start_f

clip_distance = seg_props.ClipDistance if seg_props.ClipOn else 0

try:
    if need_points:
        self._place_point_objects(
            curve_data, coll, point_template, point_settings,
            matrix_world, start_f, end_f, seg_props.EqualLengths,
            clip_distance
        )
    if need_edges:
        self._place_edge_objects(
            curve_data, coll, edge_template, edge_settings,
            matrix_world, start_f, end_f, seg_props.EqualLengths,
            clip_distance, edge_spacing
        )
except Exception as e:
    self.report({'ERROR'}, f"Error placing objects: {str(e)}")
    return {'CANCELLED'}

# Обновляем 3D View
for area in context.screen.areas:
    if area.type == 'VIEW_3D':
        area.tag_redraw()

instance_count = len(coll.objects)
coll_name = utils.get_collection_name(obj)
self.report({'INFO'}, f"Placed {instance_count} objects in collection
'{coll_name}'")
return {'FINISHED'}

def _place_point_objects(self, curve_data, coll, point_template, point_settings,
                        matrix_world, start_f, end_f, use_range, clip_distance):

```

```

        """Размещает point-объекты (по одной копии на каждую контрольную точку
кривой)."""

        for spline in curve_data.splines:

            if spline.type not in ('BEZIER', 'POLY'):
                continue

            points = spline.bezier_points if spline.type == 'BEZIER' else
spline.points

            if not points:
                continue

            spline_length = spline.calc_length()

            if spline_length == 0.0 or (clip_distance > 0 and spline_length <= 2 *
clip_distance):
                continue

            num_points = len(points)
            for i, pt in enumerate(points):
                # Получаем локальные координаты
                if spline.type == 'BEZIER':
                    coord_local = pt.co.copy()
                else:
                    coord_local = Vector(pt.co.xyz)

                # Относительная позиция точки вдоль сплайна (0..1)
                rel = 0.5 if num_points == 1 else i / (num_points - 1)

                # Клиппинг концов
                if clip_distance > 0:
                    edge_threshold = clip_distance / spline_length
                    if rel < edge_threshold or rel > (1 - edge_threshold):
                        continue

            # Диапазон

```

```

        if use_range and not (start_f <= rel <= end_f):
            continue

        # Создаём копию объекта
        new_obj = point_template.copy()
        if point_template.data:
            new_obj.data = point_template.data.copy()
        coll.objects.link(new_obj)
        new_obj.location = matrix_world @ coord_local
        new_obj.rotation_euler = point_template.rotation_euler.copy()
        new_obj.scale = point_template.scale.copy()
        utils.apply_transforms(new_obj, point_settings)
        new_obj.parent = None

    def _place_edge_objects(self, curve_data, coll, edge_template, edge_settings,
                           matrix_world, start_f, end_f, use_range, clip_distance,
                           edge_spacing):
        """Размещает edge-объекты на каждом сегменте кривой с учётом spacing и
        поворота."""
        for spline in curve_data.splines:
            if spline.type not in ('BEZIER', 'POLY'):
                continue

            pts = spline.bezier_points if spline.type == 'BEZIER' else spline.points
            if len(pts) < 2:
                continue

            spline_length = spline.calc_length()
            if spline_length == 0.0 or (clip_distance > 0 and spline_length <= 2 *
clip_distance):
                continue

            num_segments = len(pts) - 1
            use_distance = edge_spacing.use_distance

```

```

use_count = edge_spacing.use_count

for i in range(num_segments):
    # Локальные координаты концов сегмента
    if spline.type == 'BEZIER':
        p0_local = pts[i].co.copy()
        p1_local = pts[i+1].co.copy()
    else:
        p0_local = Vector(pts[i].co.xyz)
        p1_local = Vector(pts[i+1].co.xyz)

    # Относительная позиция середины сегмента (для диапазона и клиппинга)
    mid_rel = (i + 0.5) / num_segments if num_segments > 0 else 0.5

    if clip_distance > 0:
        edge_threshold = clip_distance / spline_length
        if mid_rel < edge_threshold or mid_rel > (1 - edge_threshold):
            continue

    if use_range and not (start_f <= mid_rel <= end_f):
        continue

    # Переводим в мировые координаты
    p0_world = matrix_world @ p0_local
    p1_world = matrix_world @ p1_local

    # Получаем позиции внутри сегмента (с учётом отступов)
    if use_distance:
        positions, direction = utils.get_segment_positions(
            p0_world, p1_world,
            distance=edge_spacing.edge_distance,
            offset_start=edge_spacing.offset_start,
            offset_end=edge_spacing.offset_end
        )

```

```

elif use_count:
    positions, direction = utils.get_segment_positions(
        p0_world, p1_world,
        count=edge_spacing.edge_count,
        offset_start=edge_spacing.offset_start,
        offset_end=edge_spacing.offset_end
    )
else:
    # По умолчанию: один объект в центре
    positions, direction = utils.get_segment_positions(
        p0_world, p1_world,
        offset_start=edge_spacing.offset_start,
        offset_end=edge_spacing.offset_end
    )

if not positions or direction is None:
    continue

# Кватернион поворота, чтобы объект смотрел вдоль сегмента (ось Y
вперёд, Z вверх)
track_quat = direction.to_track_quat('Y', 'Z')

for position in positions:
    new_obj = edge_template.copy()
    if edge_template.data:
        new_obj.data = edge_template.data.copy()
    coll.objects.link(new_obj)
    new_obj.location = position
    new_obj.rotation_mode = 'QUATERNION'
    new_obj.rotation_quaternion = track_quat
    new_obj.scale = edge_template.scale.copy()
    utils.apply_transforms(new_obj, edge_settings)
    new_obj.parent = None

```

```

class OBJECT_OT_clear_instances(bpy.types.Operator):
    """Удаляет все экземпляры для текущей кривой."""
    bl_idname = "object.clear_instances"
    bl_label = "Clear Instances"
    bl_description = "Remove all created instance objects for this curve"
    bl_options = {'REGISTER', 'UNDO'}

    def execute(self, context):
        obj = context.object

        if not obj or not hasattr(obj, 'main_object'):
            self.report({'WARNING'}, "Select a curve object first")
            return {'CANCELLED'}

        coll_name = utils.get_collection_name(obj)
        coll = bpy.data.collections.get(coll_name)
        count = len(coll.objects) if coll else 0

        if count > 0:
            utils.clear_instances(obj)
            self.report({'INFO'}, f"Removed {count} instance objects from '{coll_name}'")
            return {'FINISHED'}
        else:
            self.report({'INFO'}, f"No instances to remove in '{coll_name}'")
            return {'CANCELLED'}

class OBJECT_OT_clear_all_instances(bpy.types.Operator):
    """Удаляет абсолютно все коллекции Starselor и объекты в них."""
    bl_idname = "object.clear_all_instances"
    bl_label = "Clear All Instances"
    bl_description = "Remove ALL Starselor collections and objects"
    bl_options = {'REGISTER', 'UNDO'}

```

```

def execute(self, context):
    total_count = 0
    collections_count = 0
    for coll in bpy.data.collections:
        if coll.name.startswith(utils.INSTANCE_COLLECTION_PREFIX):
            total_count += len(coll.objects)
            collections_count += 1

    if total_count > 0:
        utils.clear_all_instances()
        self.report({'INFO'}, f"Removed {total_count} objects from
{collections_count} collections")
        return {'FINISHED'}
    else:
        self.report({'INFO'}, "No Starselor instances found")
        return {'CANCELLED'}

```

Список всех операторов для регистрации

```

classes = [
    CURVE_OT_toggle_handle_type,
    OBJECT_OT_place_objects,
    OBJECT_OT_clear_instances,
    OBJECT_OT_clear_all_instances,
]

```

```

def register():
    for cl in classes:
        register_class(cl)

```

```

def unregister():
    for cl in reversed(classes):
        unregister_class(cl)

```

5. Файл: panels.py

Определяет все панели интерфейса в категории «Starselor» в боковой панели 3D View.

```
"""Панели интерфейса Starselor. Группируют настройки по функциональности."""
```

```
import bpy

from bpy.utils import register_class, unregister_class
from . import utils


class MainPanel(bpy.types.Panel):
    """Главная панель: выбор шаблонов, кнопки размещения и очистки."""
    bl_label = "Starselor"
    bl_idname = "PT_MainPanel"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'

    def draw(self, context):
        layout = self.layout
        obj = context.object

        if not obj or not hasattr(obj, 'main_object'):
            layout.label(text="Select a curve object", icon='INFO')
            return

        props = obj.main_object
        coll_name = utils.get_collection_name(obj)
        coll = bpy.data.collections.get(coll_name)

        # Отображаем информацию о коллекции
        if coll:
            box = layout.box()
            box.label(text=f"Collection: {coll_name}", icon='COLLECTION_COLOR_01')
```

```

        box.label(text=f"Objects: {len(coll.objects)}")

# Выбор кривой и объектов-шаблонов
box = layout.box()
box.label(text="Curve and Objects:", icon='CURVE_DATA')
box.prop(props, "curv")
col = box.column(align=True)
col.prop(props, "ObjectsInPoints")
col.prop(props, "ObjectsInEdges")

# Кнопки действий
row = layout.row(align=True)
row.operator("object.place_objects", text="Place Objects",
icon='OBJECT_DATA')
row.operator("object.clear_instances", text="Clear", icon='X')

layout.separator()
layout.operator("object.clear_all_instances", text="Clear All Starselor
Collections", icon='TRASH')

class PanelCurveOption(bpy.types.Panel):
    """Панель опций кривой (подразделение)."""
    bl_label = "Curve Options"
    bl_idname = "PT_PanelCurveOption"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'
    bl_options = {'DEFAULT_CLOSED'}

    def draw(self, context):
        layout = self.layout
        obj = context.object

        if not obj or not hasattr(obj, 'curve_operftion'):

```

```

        return

    props = obj.curve_operftion
    col = layout.column(align=True)
    col.operator("curve.toggle_handle_type", text="Toggle Handles",
icon='MOD_CURVE')
    col.separator()
    col.prop(props, "SubdivideCurveSet")

```

```

class PanelDeletePart(bpy.types.Panel):
    """Панель видимости: скрыть point/edge объекты."""
    bl_label = "Visibility"
    bl_idname = "PT_DeletePart"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'
    bl_parent_id = 'PT_PanelCurveOption' # Дочерняя панель
    bl_options = {'DEFAULT_CLOSED'}

    def draw(self, context):
        layout = self.layout
        obj = context.object

        if not obj or not hasattr(obj, 'curve_operftion'):
            return

        props = obj.curve_operftion
        box = layout.box()
        box.label(text="Hide Objects:", icon='HIDE_OFF')
        col = box.column(align=True)
        col.prop(props, "DelCurvePoints")
        col.prop(props, "DelCurveEdges")

```

```

class EqualsPanel(bpy.types.Panel):
    """Панель диапазона размещения вдоль кривой."""

```

```

bl_label = "Placement Range"
bl_idname = "PT_Equals"
bl_space_type = 'VIEW_3D'
bl_region_type = 'UI'
bl_category = 'Starselor'
bl_options = {'DEFAULT_CLOSED'}

def draw(self, context):
    layout = self.layout
    obj = context.object

    if not obj or not hasattr(obj, 'curve_equals'):
        return

    props = obj.curve_equals
    box = layout.box()
    box.prop(props, "EqualLengths")
    if props.EqualLengths:
        col = box.column(align=True)
        col.prop(props, "StartEdge")
        col.prop(props, "EndEdge")

```

```

class ClipEqualsObj(bpy.types.Panel):
    """Панель обрезки концов кривой."""
    bl_label = "End Clipping"
    bl_idname = "PT_ClipEquals"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'
    bl_parent_id = 'PT_Equals'
    bl_options = {'DEFAULT_CLOSED'}

```

```

def draw(self, context):
    layout = self.layout
    obj = context.object

```

```

    if not obj or not hasattr(obj, 'curve_equals'):
        return

    props = obj.curve_equals
    box = layout.box()
    box.prop(props, "ClipOn")
    if props.ClipOn:
        col = box.column()
        col.prop(props, "ClipDistance")


class EdgeSpacingPanel(bpy.types.Panel):
    """Панель распределения edge-объектов: расстояние, количество, отступы."""
    bl_label = "Edge Spacing"
    bl_idname = "PT_EdgeSpacing"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'
    bl_options = {'DEFAULT_CLOSED'}

    def draw(self, context):
        layout = self.layout
        obj = context.object

        if not obj or not hasattr(obj, 'edge_spacing_props'):
            return

        props = obj.edge_spacing_props
        box = layout.box()
        col = box.column(align=True)
        col.prop(props, "use_distance")
        if props.use_distance:
            col.prop(props, "edge_distance")
        col.separator()
        col.prop(props, "use_count")
        if props.use_count:
            col.prop(props, "edge_count")

```

```

        box.separator()

        box.label(text="Offsets (in Blender units):",
icon='TRACKING_REFINE_FORWARDS')

        col = box.column(align=True)
        col.prop(props, "offset_start")
        col.prop(props, "offset_end")

class OptionObjectPanel(bpy.types.Panel):
    """Переключатель: настройки для point или edge объектов."""
    bl_label = "Object Options"
    bl_idname = "PT_OptionObject"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'
    bl_options = {'DEFAULT_CLOSED'}

    def draw(self, context):
        layout = self.layout
        obj = context.object

        if not obj or not hasattr(obj, 'curve_option_container'):
            return

        container = obj.curve_option_container
        row = layout.row()
        row.prop(container, "select_target", expand=True)

class ScalePanel(bpy.types.Panel):
    """Панель масштабирования (действует на выбранный тип объекта)."""
    bl_label = "Scale Settings"
    bl_idname = "PT_ScalePanel"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselor'

```

```

bl_parent_id = 'PT_Object'
bl_options = {'DEFAULT_CLOSED'}

def get_active_settings(self, context):
    obj = context.object
    if not obj or not hasattr(obj, 'curve_option_container'):
        return None
    container = obj.curve_option_container
    if container.select_target == '2':    # '2' = Point Object
        return container.point
    return container.edge                # '3' = Edge Object

def draw(self, context):
    layout = self.layout
    settings = self.get_active_settings(context)
    if settings is None:
        return
    box = layout.box()
    box.prop(settings, "ScaleObject")
    box.prop(settings, "NormalizationScaleObject")
    if settings.NormalizationScaleObject:
        col = box.column(align=True)
        col.prop(settings, "ScaleNormalizationObject")
    box.separator()
    box.prop(settings, "RandScaleObject")
    if settings.RandScaleObject:
        col = box.column()
        col.prop(settings, "RandScaleObjectSlider")
        box.separator()
        box.label(text="Lock Random Scale Axes:", icon='LOCKED')
        row = box.row(align=True)
        row.prop(settings, "RandScaleLockX", toggle=True)
        row.prop(settings, "RandScaleLockY", toggle=True)
        row.prop(settings, "RandScaleLockZ", toggle=True)

```

```

class RotatePanel(bpy.types.Panel):
    """Панель поворота (действует на выбранный тип объекта)."""
    bl_label = "Rotation Settings"
    bl_idname = "PT_RotatePanel"
    bl_space_type = 'VIEW_3D'
    bl_region_type = 'UI'
    bl_category = 'Starselector'
    bl_parent_id = 'PT_OptionObject'
    bl_options = {'DEFAULT_CLOSED'}

    def get_active_settings(self, context):
        obj = context.object
        if not obj or not hasattr(obj, 'curve_option_container'):
            return None
        container = obj.curve_option_container
        if container.select_target == '2':
            return container.point
        return container.edge

    def draw(self, context):
        layout = self.layout
        settings = self.get_active_settings(context)
        if settings is None:
            return
        box = layout.box()
        box.prop(settings, "NormalizationRotateObject")
        box.separator()
        box.prop(settings, "RandRotateObject")
        if settings.RandRotateObject:
            col = box.column()
            col.prop(settings, "RandRoutateObjectSlider")
            box.separator()

```

```

box.label(text="Lock Random Rotation Axes:", icon='LOCKED')
row = box.row(align=True)
row.prop(settings, "RandRotLockX", toggle=True)
row.prop(settings, "RandRotLockY", toggle=True)
row.prop(settings, "RandRotLockZ", toggle=True)

```

Список всех классов панелей для регистрации

```

classes = [
    MainPanel,
    PanelCurveOption,
    PanelDeletePart,
    EqualsPanel,
    ClipEqualsObj,
    EdgeSpacingPanel,
    OptionObjectPanel,
    ScalePanel,
    RotatePanel,
]

```

```

def register():
    for cl in classes:
        register_class(cl)

```

```

def unregister():
    for cl in reversed(classes):
        unregister_class(cl)

```